

MIAE 215 · Comprehensive Topic Guide (Part 2)

Type Casting, Modifiers, Control Flow & Microcontroller Architecture

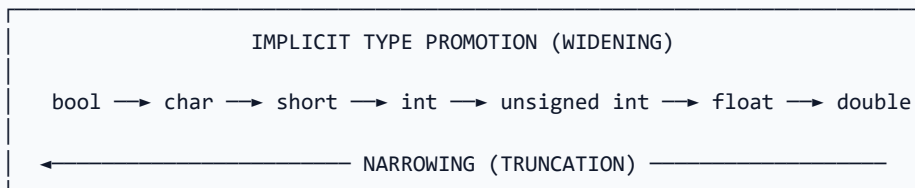
Concordia University · Department of Mechanical, Industrial & Aerospace Engineering (MIAE)

Table of Contents

1. [Type Conversions & Casting Expressions](#)
2. [Narrowing Conversions & Byte Truncation](#)
3. [The ASCII Standard & Character Arithmetic](#)
4. [Console Stream Input Mechanics: `cin`, Delimiters & Buffer Traps](#)
5. [Sentinel Loops & Clean Program Termination: `exit\(0\)`](#)
6. [C++ Type Modifiers: `short`, `long`, `unsigned`, and `const`](#)
7. [Physical Constant Definition: Exact Machine \$\pi\$ via `4 \* arctan\(1.0\)`](#)
8. [Hardware Architecture Contrast: PC \(x86/x64\) vs. Arduino \(AVR\)](#)
9. [High-Precision Scientific Output Formatting: `cout << scientific`](#)
10. [Teacher Lesson Code Walkthrough: In-Person Lecture Examples 1, 2 & 2b](#)

1. Type Conversions & Casting Expressions

In engineering applications, data from multiple disparate sources must interact: sensor ADC counts (integers), elapsed times (floating-point doubles), and serial communication packets (characters). C++ defines strict rules for converting between data types.



Implicit Promotion vs. Explicit Casting

1. Implicit Promotion (Widening):

When operands of differing types appear in an expression, C++ automatically promotes the operand with smaller bit-width to the wider type without precision loss:

```
int count = 5;
double dt = 0.25;
double total = count * dt; // count is implicitly promoted to 5.0; result is 1.25
```

2. Narrowing Conversion (Data Loss):

When copying a larger type into a smaller type, higher-order bits or fractional portions are permanently discarded:

```
double pi = 3.14159;
int truncated_pi = pi; // Fractional .14159 is discarded; truncated_pi = 3
```

3. Explicit Type Casting:

An engineer can explicitly command the compiler to convert a variable type:

- **C-Style Cast:** `(type)expression` $\implies$ e.g., `(int)x`
- **C++ Named Cast:** `static_cast<type>(expression)` $\implies$ e.g., `static_cast<int>(x)`

```
double x = 3.14;
int y = (int)x; // Explicitly forces conversion; y = 3
```

Key Takeaway for Beginners:

A cast does NOT change the original variable in RAM. In the code above, `x` remains `3.14` in its original memory location. The cast simply creates a temporary, converted copy inside a CPU register to perform the assignment.

2. Narrowing Conversions & Byte Truncation

When casting a wide variable into a narrow variable, significant data corruption can occur if the value exceeds the target's capacity.

32-bit Integer Value: 3000 (0x00000BB8)

Byte 3	Byte 2	Byte 1	Byte 0
0000 0000	0000 0000	0000 1011	1011 1000

$\uparrow$
 Casting to char (Keeps only Byte 0)
 $\downarrow$

1011 1000

 $\rightarrow$ -72 in Two's Complement!

The 3000 to `char` Truncation Trap (Teacher Demonstration)

In `variable_types2_examples/program.cpp`, Prof. Gordon illustrates this exact behavior:

```
int y = 3000;
char ch = (char)y;
cout << "(int)ch = " << (int)ch; // PRINTS: -72
```

Why Does 3000 Become -72?

1. The 32-bit binary representation of decimal 3000 is:

00000000 00000000 00001011 10111000₂

2. A signed `char` occupies only **1 byte (8 bits)**. The CPU forcibly truncates the top 24 bits, leaving only the lowest 8 bits:

10111000₂

3. Because the MSB is `1`, the number is negative in two's complement.
4. To find its magnitude, invert bits and add 1: $\text{01000111}_2 + 1 = \text{01001000}_2 = 64 + 8 = 72_{10}$
5. Therefore, the resulting value stored in `ch` is `-72`!

3. The ASCII Standard & Character Arithmetic

Computers cannot store letters or symbols; they store binary integers. The **American Standard Code for Information Interchange (ASCII)** maps numerical byte values (0 to 127) to typography.

ASCII NUMERICAL BLOCKS		
48 - 57	:	Numerical Digits '0' through '9'
65 - 90	:	Uppercase Letters 'A' through 'Z'
97 - 122	:	Lowercase Letters 'a' through 'z'
32	:	Space Character ' '
10	:	Newline Character '\n'

The Universal ASCII Offset: $\Delta = 32$

Notice the mathematical relationship between lowercase and uppercase letters:

$$'a' = 97, \quad 'A' = 65 \implies 97 - 65 = 32$$

$$'b' = 98, \quad 'B' = 66 \implies 98 - 66 = 32$$

$$'z' = 122, \quad 'Z' = 90 \implies 122 - 90 = 32$$

Every lowercase letter is located **exactly 32 positions higher** than its uppercase counterpart in memory!

```
// Converting lowercase to uppercase:
char lower = 'd';
char upper = lower - 32; // 'd' (100) - 32 = 'D' (68)

// Converting uppercase to lowercase:
char upper_char = 'M';
char lower_char = upper_char + 32; // 'M' (77) + 32 = 'm' (109)
```

Converting a Digit Character to an Actual Integer

When reading characters from serial ports or keyboard streams, digits arrive as ASCII codes:

$$'0' = 48, \quad '1' = 49, \quad \dots \quad '9' = 57$$

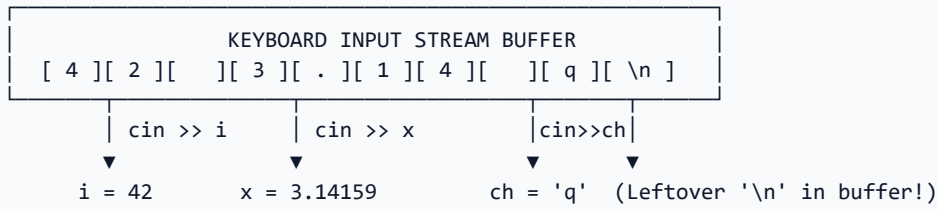
To extract the true mathematical value of a single digit character `ch`:

```
char ch = '7';
int val = ch - '0'; // '7' (55) - '0' (48) = 7
```

4. Console Stream Input Mechanics: `cin`, Delimiters & Buffer Traps

In C++, keyboard input is read through the standard input stream object `cin` (defined in `<iostream>`), utilizing the stream extraction operator (`>>`).

Keyboard Input: "42 3.14159 q [ENTER]"



Stream Delimiters

The `>>` operator skips leading whitespace and stops reading at the next **whitespace delimiter** (space, tab, or newline `\n`).

```
int i;
double x;
char ch;

cout << "Enter an int, a double, and a char: ";
cin >> i >> x >> ch; // Reads tokens separated by spaces or newlines
```

The `getchar()` Buffer Trap

A common frustration for beginners occurs when combining `cin >>` with `getchar()`:

- When a user types `42` and presses **Enter**, two items are sent to the input buffer: the characters `'4'`, `'2'`, and the newline character `'\n'`.
- `cin >> i` consumes `42`, but leaves `'\n'` lingering in the buffer.
- If you subsequently call `getchar()`, it immediately reads the leftover `'\n'` and does NOT pause the console!
- Teacher Solution:** Use two calls to `getchar()` or clear the buffer:

```
getchar(); // Consumes the leftover '\n' from cin
getchar(); // Pauses the window until the user presses Enter
```

5. Sentinel Loops & Clean Program Termination: `exit(0)`

A **sentinel value** is a special input value that signals an algorithm to terminate immediately.

In C++, immediate process termination from anywhere in the program is achieved via the `exit()` function declared in `<cstdlib>`:

- `exit(0)`: Informs the host operating system that the program concluded successfully.
- `exit(1)`: Informs the operating system of an error or abnormal termination.

```
#include <iostream>
#include <cstdlib> // Required for exit()

using namespace std;

int main() {
    char ch;
    for (int i = 0; i < 10; i++) {
        cout << "Enter command character (or '!' to abort): ";
        cin >> ch;

        if (ch == '!') {
            cout << "Termination sentinel detected. Halting program.\n";
            exit(0); // Immediately exits process
        }

        cout << "Processed command: " << ch << "\n";
    }
    return 0;
}
```

6. C++ Type Modifiers: short , long , unsigned , and const

Type modifiers alter the memory width, sign range, or mutability of fundamental data types.

C++ TYPE MODIFIERS

- signed : Default for integers & chars (+ and -)
- unsigned : Only positive numbers & zero (doubles max)
- short : Shrinks memory size (typically 2 bytes)
- long : Expands memory size (4 or 8 bytes)
- const : Read-only; forbids modification

Comprehensive Type Modifiers Table (32/64-bit PC Architecture)

Modified Type	Size (Bytes)	Range of Representable Values	Primary Engineering Usage
short int	2 bytes	−32,768 to + 32,767	Compact sensor arrays, memory-limited buffers.
unsigned short int	2 bytes	0 to 65,535	16-bit ADC measurements, DAC output commands.
int	4 bytes	−2,147,483,648 to + 2,147,483,647	Default general-purpose integer math.
unsigned int	4 bytes	0 to 4,294,967,295	Memory byte counts, microsecond timers (micros()).
long long int	8 bytes	≈ −9.22 × 10 ¹⁸ to + 9.22 × 10 ¹⁸	High-precision timestamps, astronomical counts.
unsigned char	1 byte	0 to 255	Raw hardware communication packets, RGB colors.
const double	8 bytes	Fixed constant	Physical invariants (π, gravity g, speed of light c).

The `const` Qualifier

Declaring a variable `const` instructs the compiler to lock its memory against write operations:

```
const double GRAVITY = 9.80665;
// GRAVITY = 10.0; // COMPILER ERROR: assignment of read-only variable
```

7. Physical Constant Definition: Exact Machine π via $4 \arctan(1.0)$

Many engineering textbooks define π as `3.14159`. However, hardcoding manual digits can introduce subtle floating-point truncation errors in complex orbital or aerodynamic simulations.

The Mathematical Identity

From trigonometry:

$$\tan\left(\frac{\pi}{4}\right) = 1.0 \implies \frac{\pi}{4} = \arctan(1.0) \implies \pi = 4 \times \arctan(1.0)$$

In C++ `<cmath>`, the function `atan(1.0)` computes the arctangent of 1.0 to the maximum floating-point precision of the target CPU:

```
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    const double PI = 4.0 * atan(1.0); // Exact machine-precision PI
    cout.precision(16);
    cout << "Calculated Machine PI = " << PI << "\n";
    // Output: 3.141592653589793
    return 0;
}
```

8. Hardware Architecture Contrast: PC (x86/x64) vs. Arduino (AVR)

A profound concept taught by Prof. Gordon in `variable_types2` is that **data type sizes are hardware-dependent**.

32/64-bit Desktop PC (x86/x64)

<code>sizeof(int)</code>	= 4 bytes
(-2.14B to +2.14B)	
<code>sizeof(long int)</code>	= 4/8 bytes
<code>sizeof(double)</code>	= 8 bytes

8-bit Arduino Uno (AVR ATmega328P)

<code>sizeof(int)</code>	= 2 bytes
(-32,768 to +32,767)	
<code>sizeof(long int)</code>	= 4 bytes
<code>sizeof(double)</code>	= 4 bytes!

The Embedded Microcontroller Trap

- On an 8-bit Arduino microcontroller, an `int` is only **2 bytes (16 bits)**, with an upper limit of +32,767.
- If an engineer writes a loop `for (int i = 0; i < 50000; i++)` on an Arduino, the loop will **never terminate**! The variable `i` overflows at 32,767, wraps around to -32,768, and loops infinitely!
- On an Arduino, you must explicitly use `long int` (4 bytes) for numbers exceeding 32,767.

- Furthermore, on standard 8-bit AVR Arduinos, `double` is identical to `float` (both are 4 bytes). Full 64-bit double-precision calculations do not exist without specialized 32-bit ARM microcontrollers (such as STM32 or Teensy).

9. High-Precision Scientific Output Formatting: `cout << scientific`

By default, `cout` displays only 6 significant digits and switches between decimal and scientific notation unpredictably. In engineering, consistent high-precision display is essential.

```
#include <iostream>
using namespace std;

int main() {
    double val = 3.400000000001;

    cout << scientific;      // Force scientific notation (e.g., 3.400000e+00)
    cout.precision(15);      // Display 15 decimal digits of precision

    cout << "val = " << val << "\n";
    // Output: 3.400000000001000e+00

    return 0;
}
```

10. Teacher Lesson Code Walkthrough: In-Person Lecture Examples 1, 2 & 2b

Let us examine the exact programs written and demonstrated by Prof. Gordon in the Week 2 lecture sessions:

Lecture Example 1: Multi-Type Stream Input

```
#include <iostream>
#include <cstdio>
using namespace std;

int main() {
    int i;
    double x;
    char ch;

    cout << "\ninput i ? ";
    cin >> i;

    cout << "\ninput x ? ";
    cin >> x;

    cout << "\ninput ch ? ";
    cin >> ch;

    // Single-line formatted print with comma delimiters
    cout << "\n" << i << " , " << x << " , " << ch << "\n";

    cout << "\nPress enter to continue.";
    getchar();
    return 0;
}
```

Lecture Example 2b: Interactive Filter & Case Conversion Engine

```
#include <iostream>
#include <cstdio>
#include <cstdlib> // for exit(0)

using namespace std;

int main() {
    char ch;
    int i_ch, delta, i;

    // Dynamically calculate ASCII difference between upper and lower case
    delta = (int)'a' - (int)'A'; // Exactly 32
    cout << "\ndelta = " << delta << "\n";

    for (i = 0; i < 10; i++) {
        cout << "\nInput a char ch (or '!' to exit): ";
        cin >> ch;

        // Check for termination sentinel
        if (ch == '!') {
            cout << "Exiting program via exit(0)...\n";
            exit(0);
        }

        // Convert char to integer for ASCII bounds evaluation
        i_ch = (int)ch;

        // Test if character is inside lowercase ASCII domain [97, 122]
        if ((97 <= i_ch) && (i_ch <= 122)) {
            i_ch = i_ch - delta; // Subtract 32 to convert to uppercase
            ch = (char)i_ch;     // Cast back to char
            cout << "Converted upper case ch = " << ch << "\n";
        } else {
            cout << "Character is already uppercase or non-alphabetic: " << ch << "\n";
        }
    }

    return 0;
}
```

Detailed Trace of Lecture Example 2b

Iterat ion	Keyboard Input	Variable ch	i_ch (ASCII)	Lowercase Condition (97 <= i_ch <= 122)	Transformation	Final Output
0	'g'	'g'	103	TRUE	103 − 32 = 71 ⇒ 'G'	Converted upper case ch = G
1	'R'	'R'	82	FALSE	None (Already uppercase)	Character is already uppercase: R
2	'5'	'5'	53	FALSE	None (Numeric digit)	Character is already uppercase: 5
3	'!'	'!'	33	Sentinel triggered	Calls exit(0)	Program halts immediately